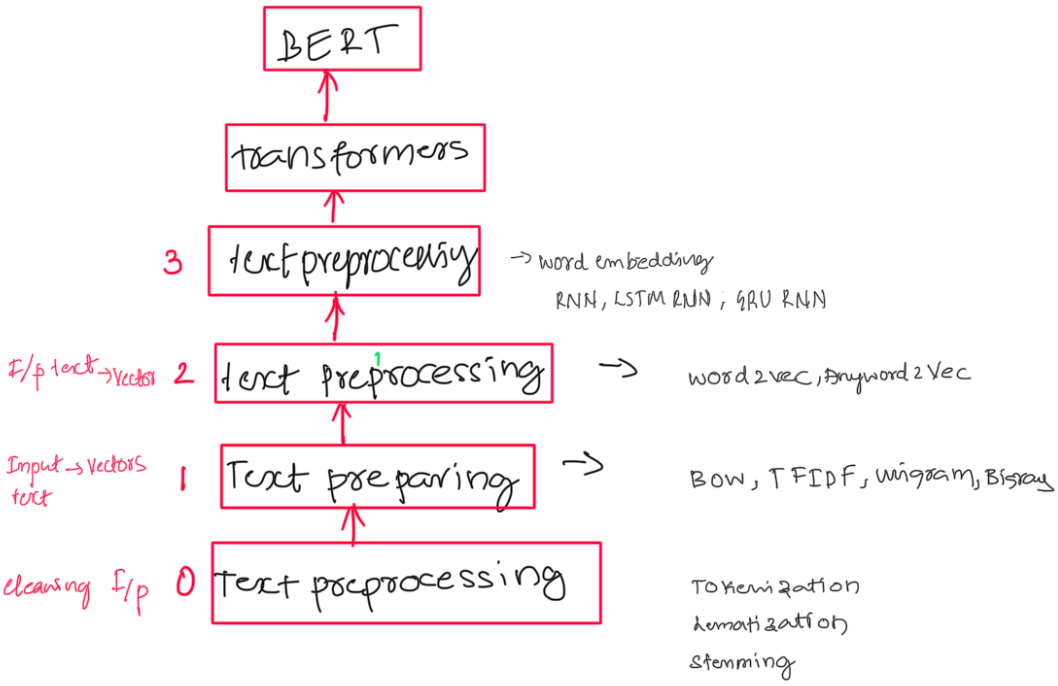
* NLP

* Roadmap of NLP



* tokenization using NLTK

corpus — Paragraph

Documents — sentence

unique words — word

1) from nltk.tokenize import sent_tokenize

paragraph → sentences

2) from nltk.tokenize import word_tokenize

paragraph → words

3) from nltk.tokenize import wordpunct_tokenize

word's → 'word', ''', 's'

* tokenize punctutation's also.

* Stemming

* reduce a word to its word stem

eating → eat
waiting → wait

- used in classification problem
  1. Reviews
     ↳ eating, eaten → eat

1) Porter Stemmer
   > from nltk.stem import PorterStemmer
   > stemming = PorterStemmer()
   > stemming.stem(words)
     ↳ function //

* disadvantage
  - can't work with complicated words

  'congratulations' → 'congtul.' x

2) Regexpstemmer class
   > from nltk.stem import RegexpStemmer

- takes a regular expression and removes the prefix and sufix that matches the expression

  > RegexpStemmer ('ing$|s$|e$|able$')

  $ = end of the word
  | = or operation

if any word contains ing $

- this removes the matching letters in the word to be stemmed

3) Snowball stemmer

> from nltk.stem import Snowball

- this performs better than the above and also provides multilingual support

* Lemmatization

1) wordnet lemmatizer

- lemmatization technique is like stemming
- the output we will get after is called "lemma"
- its not stem of a word but a meaningfull word that means same thing

```
> from nltk.stem import WordNetLemmatizer
> l = WordNetLemmatizer()
> l.lematize("going", pos='v')
```

function just like stem

o/p
> go.

Postag

Noun – n
Verb – v
adjective – a
adverb – r

pos = 'v'

* takes more time then stem

Usecase

- Q and A
- chatbots
- text summrization

* part of Speech tagging

> uses nltk.pos_tag("word")

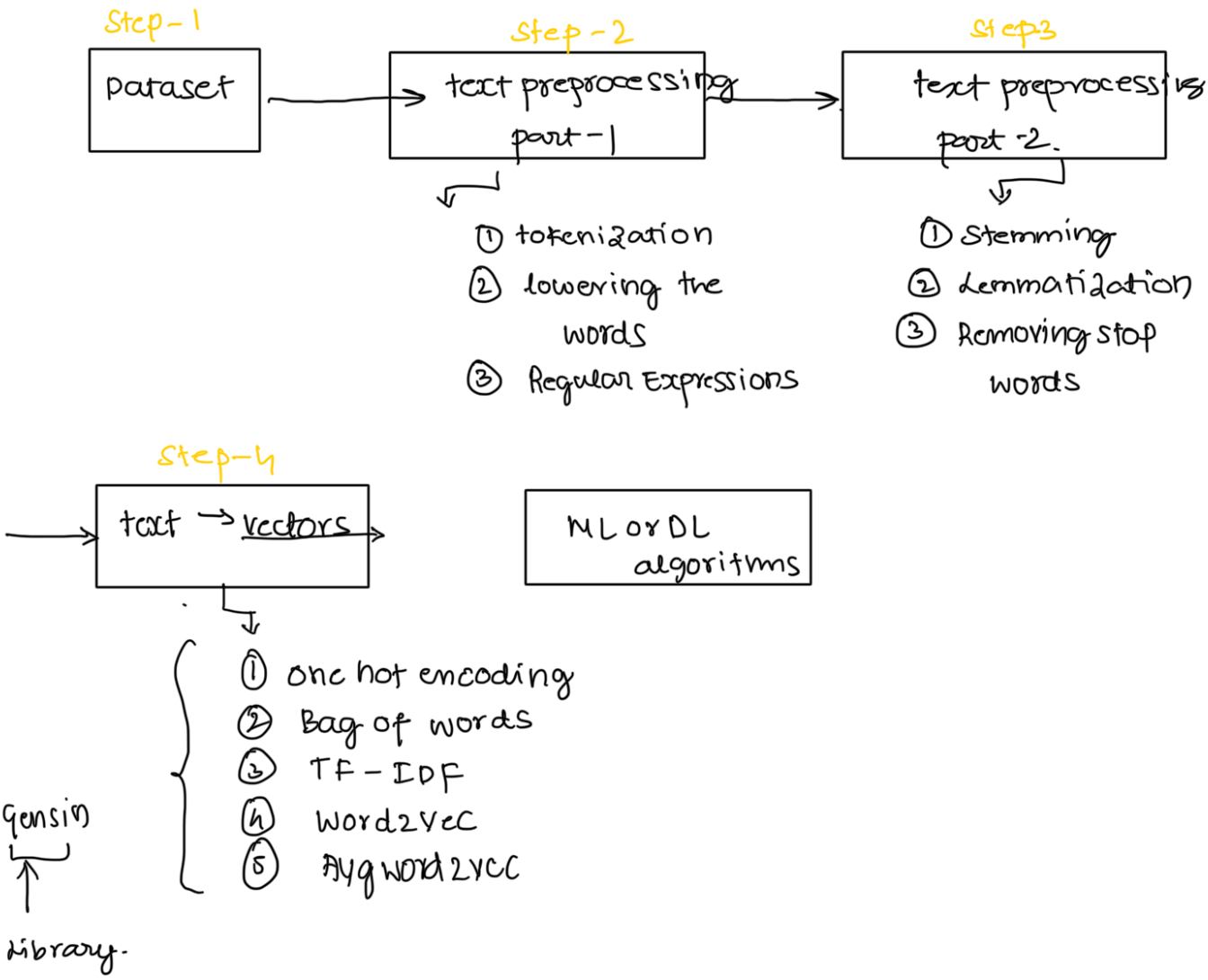
* tells us the part of the speech of the words

* named entity recognition:

* its a NLP tasks used for recognizing and classify important entities in a sentences.

→ what did we learn??

* TEXT PREprocessing



① one hot encoding:

text

D1, the food is good

D2, the food is bad

D3, Pizza is amazing

Vocabulary {unique words}

$v = 7$

the food is good bad Pizza amazing

$$
\begin{aligned}
&[1 \quad 0 \quad 0 \quad 0 \quad 0 \quad 0 \quad 0] \\
\text{food.} = &[0 \quad 1 \quad 0 \quad 0 \quad 0 \quad 0 \quad 0] \\
\text{is.} = &[0 \quad 0 \quad 1 \quad 0 \quad 0 \quad 0 \quad 0] \\
\text{good.} = &[0 \quad 0 \quad 0 \quad 1 \quad 0 \quad 0 \quad 0]
\end{aligned}
$$

So, D1 can be represented by:

$\boxed{\text{text} \rightarrow \text{vectors}}$

D2 = " the food is bad "

$$
D1 = \left[\begin{array}{l} [1\ 0\ 0\ 0\ 0\ 0\ 0], \\ [0\ 1\ 0\ 0\ 0\ 0\ 0], \\ [0\ 0\ 1\ 0\ 0\ 0\ 0], \\ [0\ 0\ 0\ 0\ 1\ 0\ 0] \end{array} \right]
$$

4×7

vector representation of D1 document

D1 = " the food is good"

$$
D2 = \left[\begin{array}{l} [1\ 0\ 0\ 0\ 0\ 0\ 0], \\ [0\ 1\ 0\ 0\ 0\ 0\ 0], \\ [0\ 0\ 1\ 0\ 0\ 0\ 0], \\ [0\ 0\ 0\ 1\ 0\ 0\ 0] \end{array} \right]
$$

4×7

vector representation of D2 representation

$$
D3 = \left[\begin{array}{l} [1\ 0\ 0\ 0\ 0\ 0\ 0], \\ [0\ 0\ 0\ 0\ 0\ 1\ 0], \\ [0\ 0\ 0\ 0\ 0\ 0\ 1] \end{array} \right]
$$

3×7 → variation in the size of the vector

here

Advantages

disadvantages

* Easy to implement in python

[sklearn, pd.get_dummies()]

* Sparse matrix → overfitting

* for ml algorithm we need Fixed size Input.

* no semantic meaning is captured.

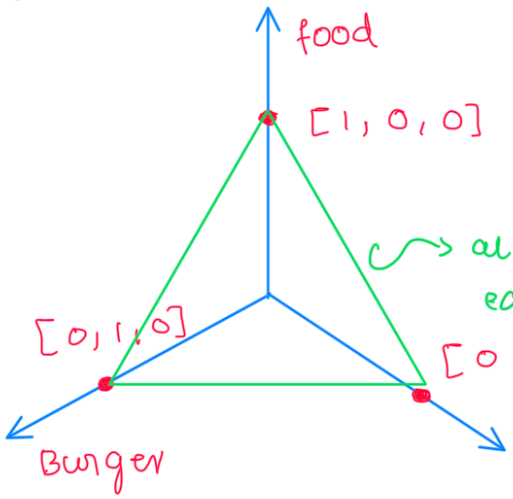
let's take "pizza", "Burger", "food"

$$\begin{bmatrix} 1 & 0 & 0 \end{bmatrix}$$

$$\begin{bmatrix} 0 & 1 & 0 \end{bmatrix}$$

$$\begin{bmatrix} 0 & 0 & 1 \end{bmatrix}$$

if we try to calculate cosine similarity between them



→ all this words are in equal distance and can't tell the difference between the words

• it is considering all the words equal

so, semantic analysis ⇒ understanding the meaning of the sentence

semantic analysis →

1. what is realationship b/w word?

"dog chased cat"

dog → Agent (who performs action)

chased → Action

2. what does the sentence mean? — cat → object (who receives action)

* out of vocabulary

vocabulary {unique words}

v = 7

the, food, is, good, bad, pizza, amazing,

if the sentence was....

"burger is bag."

?? (burger) WTF!

??? → out of vocabulary

we can't represent unless it is in the vocabulary

* Bag of words

dataset

I/p	o/p
he is a good boy	1 → positive (not negative)
he is a good girl	1
Boy and girl are good	1

and → stopword

lower all the words case

remove stop words

vocabulary	frequency
good	3

boy 2

girl 2

	good	boy	girl	O/p
S1	1	1	0	1
S2	1	0	1	1
S3	1	1	1	1

if good takes place two times then we can update it 2 or 3

* in binary bag of words we can't update it to 2 or 3

Bag of words

* Advantage
  * simple and intiutive
  * Fixed sized of Input

* disadvantage
  * Sparse matrix is still there
  * ordering of sentence changes.
  * out of vocabulary.
  *